DSA Interview Cheatsheet

Trees, Linked Lists, Backtracking & Recursion Patterns

• Core Concepts

References - The Foundation

CRITICAL: Objects are passed by reference!
LinkedList Example:

```
ListNode dummy = new ListNode(0);
ListNode node = dummy;  // Same object!

node.val = 5;
System.out.println(dummy.val);  // 5
```

TreeNode Example:

```
TreeNode root = /* tree */;
Queue<TreeNode> q = new LinkedList<>();
q.add(root);  // Adds reference, not copy!

TreeNode node = q.poll();
// node points to SAME object as root
```

Array Example:

```
int[] arr = new int[1];
helper(arr);  // Pass reference

void helper(int[] arr) {
    arr[0] = 5;  // Modifies THE array
}
```

Remember:
✓ Objects (arrays, nodes) reference
× Primitives (int, boolean) copy

Why int Parameter Fails

This DOESN'T work:

```
void main() {
    int diameter = 0;
    helper(diameter);
    return diameter;  // Still 0!
}

void helper(int diameter) {
    diameter = 5;  // Only changes COPY
}
```

Each call gets its OWN copy:

```
main(): diameter = 0 (original)
helper(): diameter = 5 (copy, destroyed)
Back to main(): diameter = 0 (unchanged)
```

This WORKS - Array:

```
void main() {
    int[] diameter = new int[1];  // ONE
    array
    helper(diameter);  // Pass reference
    return diameter[0];  // Updated!
}

void helper(int[] diameter) {
    diameter[0] = 5;  // Modifies THE array
}
```

Key Insight:
Only ONE array created!
All recursive calls share it!

Tree Types

Binary Tree: Each node has ≤ 2 children

```
class TreeNode {
int val;
TreeNode left, right;
}
```

N-ary Tree: Each node has any # of children

```
class Node {
int val;
List<Node> children;
}
```

TreeNode like ListNode:
`ListNode.next` one reference
`TreeNode.left/right` two references
Both use references to connect!

DFS vs BFS

DFS (Depth-First Search):
✓ Goes DEEP first
✓ Uses: Stack or Recursion
✓ Traversals: Preorder, Postorder, Inorder
✓ Space: O(height)
✓ Good for: Path problems, exhaustive search

BFS (Breadth-First Search):
✓ Goes WIDE first (level by level)
✓ Uses: Queue (FIFO)
✓ Traversal: Level Order
✓ Space: O(width)
✓ Good for: Shortest path, min depth

Data Structures Quick Reference

Stack (DFS): LIFO

```
Stack<Node> s = new Stack<>();
s.push(node); s.pop();
```

Queue (BFS): FIFO

```
Queue<Node> q = new LinkedList<>();
q.offer(node); q.poll();
```

List of Lists:

```
List<List<Integer>> result = new ArrayList<>();
result.add(new ArrayList<>()); // Empty list
System.out.println(result); // [[]]
```

HashMap with List:

```
Map<K, List<V>> map = new HashMap<>();
map.computeIfAbsent(key, k -> new
ArrayList<>()).add(val);
```

Char to Int:

```
char digit = '2';
int value = digit - '0'; // = 2
```

⋆ Backtracking Fundamentals

Backtracking Pattern

Core Template:

```
void backtrack(state, choices) {
    if (is_solution(state)) {
        save_solution(state);
        return;
    }

    for (each choice in choices) {
        make_choice(choice);       // 1
        backtrack(new_state);      // 2
        undo_choice(choice);       // 3
    }
}
```

Three Steps:

1. Make a choice
2. Recurse (explore)
3. Undo (backtrack)

When to Use:
✓ Need ALL solutions
✓ "All combinations/permutations"
✓ Decision tree exploration

Decision Tree Concept

Think of it as exploring a tree:

```
            ""
          / | \
         a  b  c
        / \ |  |
      ... choices at each level
```

Each path = one solution
Backtracking = DFS on decision tree

▷ Binary Tree Level Order

Problem: Return values level by level
Time: O(n) **Space:** O(w) $w = max\ width$
Key Pattern: BFS with Level Tracking

```
public List<List<Integer>> levelOrder(
    TreeNode root) {
    List<List<Integer>> result =
        new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        int levelSize = q.size();  // CRITICAL!

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = q.poll();
            level.add(node.val);

            if (node.left != null)
                q.add(node.left);
            if (node.right != null)
                q.add(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Why Capture q.size()?
Queue size changes as we add children!
Capture size BEFORE loop = process exactly one level
Example Trace:

```
Tree:   1
       / \
      2   3

Initial: q = [1]
Iter 1: levelSize = 1
  Poll 1, add 2,3
  level = [1], q = [2,3]

Iter 2: levelSize = 2 (captured!)
  Poll 2, poll 3
  level = [2,3], q = []

Result: [[1], [2,3]]
```

Without capturing size:

```
// WRONG!
for (int i = 0; i < q.size(); i++) {
    // q.size() keeps growing!
    // Processes multiple levels
}
```

Why new ArrayList<>() each iteration?
Each level needs its own list!
`result.add(level)` adds reference
If we reuse, all would point to same list!

▷ Binary Tree Diameter

Problem: Longest path between any two nodes (in edges)
Time: O(n) **Space:** O(h)
Key Insight:
At each node: diameter = leftHeight + rightHeight
Approach 1: Instance Variable

```
class Solution {
    int diameter = 0;  // Track maximum

    public int diameterOfBinaryTree(
        TreeNode root) {
        height(root);
        return diameter;
    }

    private int height(TreeNode node) {
        if (node == null) return 0;

        int leftHeight = height(node.left);
        int rightHeight = height(node.right);

        // Update global max
        diameter = Math.max(diameter,
            leftHeight + rightHeight);

        // Return height for parent
        return 1 + Math.max(leftHeight,
            rightHeight);
    }
}
```

Approach 2: Array Parameter

```
public int diameterOfBinaryTree(
    TreeNode root) {
    int[] diameter = new int[1];  // ONE array
    height(root, diameter);
    return diameter[0];
}

private int height(TreeNode node,
    int[] diameter) {
    if (node == null) return 0;

    int leftHeight = height(node.left,
        diameter);
    int rightHeight = height(node.right,
        diameter);

    // All calls modify SAME array!
    diameter[0] = Math.max(diameter[0],
        leftHeight + rightHeight);

    return 1 + Math.max(leftHeight,
        rightHeight);
}
```

Why Need Global Variable or Array?
Function must return TWO things:
1. Height (for parent's calculation)
2. Diameter (the answer)
Can't return both track diameter separately!
Why Array Works:

```
Only ONE array created!
All recursive calls get REFERENCE to it
All modify SAME array
```

Instance Variable vs Array:

Instance Var	Array
Cleaner code	More explicit
No extra param	Shows dependency
Must reset manually	Auto reset

▷ Letter Combinations

Problem: Phone number to letter combinations
Mapping: 2abc, 3def, ..., 9wxyz
Time: $O(4^n)$ **Space:** O(n)
Approach: Backtracking

```
String[] digitToChar = {
    "", "", "abc", "def", "ghi", "jkl",
    "mno", "pqrs", "tuv", "wxyz"
};

public List<String> letterCombinations(
    String digits) {
    List<String> res = new ArrayList<>();
    if (digits.isEmpty()) return res;
    backtrack(0, "", digits, res);
    return res;
}

private void backtrack(int i, String cur,
    String digits, List<String> res) {
    // Base case: built complete combination
    if (cur.length() == digits.length()) {
        res.add(cur);
        return;
    }

    // Get letters for current digit
    String chars =
        digitToChar[digits.charAt(i) - '0'];

    // Try each letter
    for (char c : chars.toCharArray()) {
        backtrack(i + 1, cur + c, digits, res);
    }
}
```

Why No Explicit Undo?
Strings are immutable!
`cur + c` creates new string automatic backtrack
Trace for "23":

```
bt(0, "")
  Try 'a': bt(1, "a")
    Try 'd': bt(2, "ad")  save "ad"
    Try 'e': bt(2, "ae")  save "ae"
    Try 'f': bt(2, "af")  save "af"
  Try 'b': bt(1, "b")
    ... generates "bd", "be", "bf"
  Try 'c': bt(1, "c")
    ... generates "cd", "ce", "cf"
```

▷ Permutations

Problem: All arrangements of array elements
Example: [1,2,3] [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
Time: O(n!) **Space:** O(n)
Approach: Backtracking with Used Array

```
public List<List<Integer>> permute(
    int[] nums) {
    List<List<Integer>> result =
        new ArrayList<>();
    boolean[] used = new boolean[nums.length];
    backtrack(nums, new ArrayList<>(),
        used, result);
    return result;
}

private void backtrack(int[] nums,
    List<Integer> current, boolean[] used,
    List<List<Integer>> result) {
    // Base case: used all elements
    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    // Try each unused element
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;

        // Make choice
        current.add(nums[i]);
        used[i] = true;

        // Recurse
        backtrack(nums, current, used, result);

        // Undo choice
        current.remove(current.size() - 1);
        used[i] = false;
    }
}
```

Why used[] needed?
Prevents using same element multiple times!
Without it: [1,1,1] instead of [1,2,3]
Decision Tree:



▷ Subsets

Problem: All possible subsets (power set)
Example: [1,2,3] [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]
Time: $O(2^n)$ **Space:** O(n)
Approach 1: Include/Exclude

```
public List<List<Integer>> subsets(
    int[] nums) {
    List<List<Integer>> result =
        new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(),
        result);
    return result;
}

private void backtrack(int[] nums,
    int index, List<Integer> current,
    List<List<Integer>> result) {
    // Base case: processed all elements
    if (index == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    // Include current element
    current.add(nums[index]);
    backtrack(nums, index + 1, current,
        result);
    current.remove(current.size() - 1);

    // Exclude current element
    backtrack(nums, index + 1, current,
        result);
}
```

Approach 2: Build from Position (Cleaner)

```
private void backtrack(int[] nums,
    int start, List<Integer> current,
    List<List<Integer>> result) {
    // Every state is valid - save it!
    result.add(new ArrayList<>(current));

    // Try adding elements from start onwards
    for (int i = start; i < nums.length; i++){
        current.add(nums[i]);
        backtrack(nums, i + 1, current, result);
        current.remove(current.size() - 1);
    }
}
```

Key Difference:
Approach 1: Binary decision (include/exclude)
Approach 2: Save at every node, iterate choices

♡ Tree Traversals

Traversal Orders

Given tree:

```
    1
   /|\
  2 3 4
 /
5
```

Preorder: Root Children
Result: [1, 2, 5, 3, 4]
Postorder: Children Root
Result: [5, 2, 3, 4, 1]
Level Order: Level by level
Result: [[1], [2, 3, 4], [5]]

Preorder Recursive:

```
private void preorder(Node node) {
    if (node == null) return;
    result.add(node.val);  // Process first
    for (Node child : node.children) {
        preorder(child);
    }
}
```

Postorder Recursive:

```
private void postorder(Node node) {
    if (node == null) return;
    for (Node child : node.children) {
        postorder(child);
    }
    result.add(node.val);  // Process last
}
```

Level Order BFS:

```
Queue<Node> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
    int levelSize = q.size();  // KEY!
    List<Integer> level = new ArrayList<>();

    for (int i = 0; i < levelSize; i++) {
        Node node = q.poll();
        level.add(node.val);
        for (Node child : node.children) {
            q.offer(child);
        }
    }
    result.add(level);
}
```

▷ Max/Min Depth

Max Depth - Recursive:

```
public int maxDepth(Node root) {
    if (root == null) return 0;

    int max = 0;
    for (Node child : root.children) {
        max = Math.max(max, maxDepth(child));
    }
    return max + 1;
}
```

Min Depth - BFS (Optimal!):

```
public int minDepth(TreeNode root) {
    if (root == null) return 0;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;

    while (!q.isEmpty()) {
        depth++;
        int size = q.size();

        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();

            // First leaf = min depth!
            if (node.left == null &&
                node.right == null) {
                return depth;
            }

            if (node.left != null)
                q.offer(node.left);
            if (node.right != null)
                q.offer(node.right);
        }
    }
    return depth;
}
```

Why BFS for Min Depth?
Explores level-by-level first leaf is minimum!
DFS must explore all paths.

▷ Validate BST

Problem: Check if tree is valid Binary Search Tree
BST Property:
ALL left subtree < node < ALL right subtree
Time: O(n) **Space:** O(h)
Approach: Min/Max Bounds

```
public boolean isValidBST(TreeNode root) {
    return valid(root, Long.MIN_VALUE,
        Long.MAX_VALUE);
}

public boolean valid(TreeNode node,
    long left, long right) {
    if (node == null) return true;

    // Check if node in valid range
    if (!(left < node.val &&
          node.val < right)) {
        return false;
    }

    // Left subtree: all < node.val
    // Right subtree: all > node.val
    return valid(node.left, left, node.val) &&
           valid(node.right, node.val, right);
}
```

Why Long not Integer?
`Long.MIN_VALUE < Integer.MIN_VALUE`
`Long.MAX_VALUE > Integer.MAX_VALUE`
All int values fit inside long range!
Example:

```
Tree: Integer.MIN_VALUE as node
With int bounds: MIN < MIN? NO
With long bounds: Long.MIN < Int.MIN? YES
```

Bounds Propagation:

```
        5 (-, +)
       / \
(-,5)/   \(5,+)
    /     \
   3       7 (5, +)
          / \
   (5,7) /   \ (7,+)
        6     8
```

▷ Lowest Common Ancestor

Problem: Find deepest common ancestor of two nodes
Time: O(n) **Space:** O(h)
Key Insight:
If p in left AND q in right (or vice versa)
Current node IS the LCA!

```
public TreeNode lowestCommonAncestor(
    TreeNode root, TreeNode p, TreeNode q) {
    // Base case 1: empty tree
    if (root == null) return null;

    // Base case 2: found target
    // (even if other is below!)
    if (root == p || root == q) return root;

    // Search both subtrees
    TreeNode left = lowestCommonAncestor(
        root.left, p, q);
    TreeNode right = lowestCommonAncestor(
        root.right, p, q);

    // Both non-null = split case
    if (left != null && right != null) {
        return root;  // This is LCA
    }
```

```
    // Otherwise, return non-null one
    // (or null if both null)
    return left != null ? left : right;
}
```

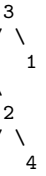
Why return when we find p or q?

- If other node is below this IS LCA
- If other node elsewhere parent will know
- Don't need to search further down!

Three Cases:

1. Both in left subtree return left
2. Both in right subtree return right
3. Split between subtrees return root

Example:



```
LCA(7, 4) = 2 (split at node 2)
LCA(5, 1) = 3 (split at root)
LCA(5, 4) = 5 (5 is ancestor of 4)
```

Trace for LCA(7, 4):

```
At node 7: root == p, return 7
At node 4: root == q, return 4
At node 2:
  left = 7 (non-null)
  right = 4 (non-null)
  BOTH non-null  return 2 (LCA!)
```

▷ Binary Tree Right Side View

Problem: Values visible from right side (rightmost node at each level)
Time: O(n) **Space:** O(w)

```
public List<Integer> rightSideView(
    TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int qLen = q.size();
        TreeNode rightSide = null;

        for (int i = 0; i < qLen; i++) {
            TreeNode node = q.poll();
            if (node != null) {
                rightSide = node;  // Keep updating
                q.offer(node.left);
                q.offer(node.right);
            }
        }

        if (rightSide != null) {
            res.add(rightSide.val);
        }
    }
    return res;
}
```

Why Keep Updating Works:
Process nodes leftright in level
Last update = rightmost node!
Alternative - More Explicit:

```
for (int i = 0; i < qLen; i++) {
    TreeNode node = q.poll();

    if (i == qLen - 1) {  // Last in level
        res.add(node.val);
    }

    if (node.left != null) q.offer(node.left);
    if (node.right != null) q.offer(node.right);
}
```

▷ Invert Binary Tree

Problem: Swap left and right children of ALL nodes
Time: O(n) **Space:** O(h)

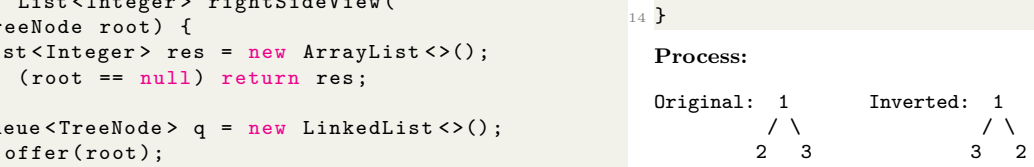
```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;

    // Swap children
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;

    // Recursively invert subtrees
    invertTree(root.left);
    invertTree(root.right);

    return root;
}
```

Process:



○ Reverse Linked List

Problem: Reverse a singly linked list
Time: O(n) **Space:** O(n) *call stack*

```
public ListNode reverseList(ListNode head) {
    // Base case
    if (head == null || head.next == null)
        return head;

    // Recursive call - returns new head
    ListNode newHead = reverseList(head.next);

    // Reverse the pointer
    head.next.next = head;

    // Break the chain
    head.next = null;

    return newHead;
}
```

Key Understanding:
`newHead` is ALWAYS the same (last node)!
`head` is DIFFERENT in each frame!
All frames return the SAME newHead!
Trace for 123:

```
Call: reverse(1)
  Call: reverse(2)
    Call: reverse(3)
      Base: return 3
    newHead = 3
    2.next.next = 2   32
    2.next = null     32null
    return 3
  newHead = 3 (same!)
  1.next.next = 1     321
  1.next = null       321null
  return 3 (same!)

Final: 321null
```

Why head.next = null needed?

```
Without: Creates cycle (12)
With: Breaks old link (1null)
```

▷ Graph: Number of Islands

Problem: Count connected components of 1's
Time: O(mŒn) **Space:** O(mŒn)

```
public int numIslands(char[][] grid) {
    if (grid == null) return 0;
    int count = 0;

    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++){
            if (grid[i][j] == '1') {
                dfs(grid, i, j);  // Mark island
                count++;  // Found new island
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int i, int j) {
    // Boundary check
    if (i < 0 || i >= grid.length ||
        j < 0 || j >= grid[0].length ||
```

```
        grid[i][j] == '0') return;

    grid[i][j] = '0';  // Mark visited

    // Explore 4 directions
    dfs(grid, i + 1, j);  // down
    dfs(grid, i - 1, j);  // up
    dfs(grid, i, j + 1);  // right
    dfs(grid, i, j - 1);  // left
}
```

Key Insight:
Each DFS marks entire island count only when starting new DFS

▷ Graph: Max Area of Island

Problem: Find largest connected component
Time: O(mŒn) **Space:** O(mŒn)

```
public int maxAreaOfIsland(int[][] grid) {
    int maxArea = 0;

    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++){
            if (grid[i][j] == 1) {
                int area = dfs(grid, i, j);
                maxArea = Math.max(maxArea, area);
            }
        }
    }
    return maxArea;
}

private int dfs(int[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length ||
        j < 0 || j >= grid[0].length ||
        grid[i][j] == 0) {
        return 0;
    }

    grid[i][j] = 0;  // Mark visited

    // Count this cell + all connected
    return 1 + dfs(grid, i + 1, j)
             + dfs(grid, i - 1, j)
             + dfs(grid, i, j + 1)
             + dfs(grid, i, j - 1);
}
```

Return Area:
Each cell contributes 1 + sum of neighbors

▷ Meeting Rooms

Problem: Check if person can attend all meetings
Time: O(n log n) **Space:** O(1)

```
public boolean canAttendMeetings(
    List<Interval> intervals) {
    // Sort by start time
    intervals.sort((a,b) -> a.start - b.start);

    // Check consecutive meetings
    for (int i = 1; i < intervals.size(); i++) {
        if (intervals.get(i).start <
            intervals.get(i-1).end) {
            return false;  // Overlap!
        }
    }
    return true;
}
```

Why Sort?
After sorting, only need to check consecutive pairs!
If i doesn't overlap i-1, can't overlap earlier ones.

♢ Common Patterns

Global Variable vs Array vs Return

Use Global Variable when:

- Need to track max/min/count
- Must return something else (like height)
- Example: Binary Tree Diameter

Use Array Parameter when:

- Want to avoid instance variables
- Need automatic reset between calls
- More explicit dependencies

Use Return Value when:

- Can return the answer directly
- Example: LCA (returns the node)
- Example: Max Depth (returns the value)

Can't Use Primitive Parameter:

```
// WRONG - each call gets COPY
void helper(int count) {
    count++;  // Only changes copy
}

// CORRECT - use array (reference)
void helper(int[] count) {
    count[0]++;  // Modifies THE array
}
```

HashMap with Multiple Values

Store multiple values per key:

```
Map<Integer, List<Character>> map =
    new HashMap<>();

// Add value
map.computeIfAbsent(key,
    k -> new ArrayList<>()).add(val);

// Why computeIfAbsent?
// Creates list if not exists, then adds
// No manual null checks needed!
```

Use Cases:

- Group Anagrams (string list of anagrams)
- Graph adjacency list (node neighbors)
- Index mapping (value list of indices)

Character/Digit Conversion

Char to Int:

```
char digit = '5';
int value = digit - '0';  // = 5

// Why this works:
// '0' has ASCII 48, '5' has ASCII 53
// 53 - 48 = 5
```

Use as Array Index:

```
String[] phoneMap = {"", "", "abc", ...};
char digit = '2';
String letters = phoneMap[digit - '0'];
// = phoneMap[2] = "abc"
```

When to Use Level Parameter

DON'T need level:

- Preorder/Postorder (flat list)
- Max/Min depth (single value)

DO need level:

- Level order with DFS (nested lists)
- When result depends on depth

Why?

```
// Flat output - just append
result.add(node.val);

// Nested output - need index
result.get(level).add(node.val);
//
//   Which sublist?
```

BFS Level Order Template

```
Queue<Node> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
    int levelSize = q.size();  // Capture!

    for (int i = 0; i < levelSize; i++) {
        Node node = q.poll();
        // Process node

        // Add children
        for (Node child : node.children) {
            q.offer(child);
        }
    }
}
```

Bounds Validation Template

```
boolean valid(TreeNode node,
    long min, long max) {
    if (node == null) return true;

    // Check current node
    if (!(min < node.val &&
          node.val < max)) {
        return false;
    }

    // Recurse with tighter bounds
    return valid(node.left, min, node.val) &&
           valid(node.right, node.val, max);
}
```

♣ Code Templates

Backtracking Template

```
void backtrack(state, choices) {
    if (is_complete(state)) {
        save(state);
        return;
    }

    for (each choice in choices) {
        make_choice();
        backtrack(new_state, new_choices);
        undo_choice();  // KEY!
    }
}
```

DFS with Global Variable

```
class Solution {
    int result = 0;  // Track answer

    public int problem(TreeNode root) {
        helper(root);
        return result;
    }

    private int helper(TreeNode node) {
        if (node == null) return 0;

        int left = helper(node.left);
        int right = helper(node.right);

        // Update global
        result = Math.max(result, left +
    right);

        // Return something else
        return 1 + Math.max(left, right);
    }
}
```

× Common Mistakes

Backtracking Mistakes

× Missing undo step

```
// WRONG
current.add(choice);
backtrack(current);
// Missing: current.remove(...)

// CORRECT
current.add(choice);
backtrack(current);
current.remove(current.size() - 1);
```

× Not making copy when saving

```
// WRONG
result.add(current);  // Reference!

// CORRECT
result.add(new ArrayList<>(current));
```

BFS Mistakes

× Using queue.size() in loop

```
// WRONG - size changes!
for (int i = 0; i < q.size(); i++) {...}

// CORRECT
int size = q.size();
for (int i = 0; i < size; i++) {...}
```

× Adding null to queue

```
// WRONG
q.offer(node.left);  // May be null!

// CORRECT
if (node.left != null) q.offer(node.left);
```

× Creating new list OUTSIDE loop

```
// WRONG
List<Integer> level = new ArrayList<>();
while (!q.isEmpty()) {
    // Reusing same list!
}

// CORRECT
while (!q.isEmpty()) {
    List<Integer> level = new ArrayList<>();
    // New list each iteration
}
```

Tree Mistakes

× Only checking immediate children

```
// WRONG for BST validation
if (node.val < node.left.val) ...

// CORRECT - use bounds
if (!(min < node.val && node.val < max))
```

× Min depth without leaf check

```
// WRONG
minDepth = Math.min(minDepth, height);

// CORRECT
if (node.left == null && node.right == null)
    {
    minDepth = Math.min(minDepth, height);
}
```

× Using int parameter for tracking

```
// WRONG - primitives are copied!
void helper(int count) {
    count++;  // Only changes local copy
}

// CORRECT - use array or instance var
int count = 0;  // Instance variable
void helper() {
    count++;  // Modifies shared variable
}
```

Complexity Reference

Problem	Time	Space
Tree DFS	O(n)	O(h)
Tree BFS	O(n)	O(w)
Level Order	O(n)	O(w)
Diameter	O(n)	O(h)
Letter Comb	$O(4^n)$	O(n)
Permutations	O(n!)	O(n)
Subsets	$O(2^n)$	O(n)
Validate BST	O(n)	O(h)
LCA	O(n)	O(h)
Graph DFS	O(V+E)	O(V)

Where:
n = nodes, h = height, w = width
V = vertices, E = edges

♠ Learning Path

Phase 1: Fundamentals

1. Tree traversals (589, 590, 144)
2. Level order (102, 429)
3. Max/Min depth (104, 111)

Phase 2: Backtracking

1. Letter Combinations (17)
2. Subsets (78)
3. Permutations (46)
4. Combination Sum (39)

Phase 3: Advanced Trees

1. Diameter (543)
2. Validate BST (98)
3. LCA (236)
4. Right Side View (199)
5. Path Sum (112, 113)

Phase 4: Graphs

1. Number of Islands (200)
2. Max Area of Island (695)
3. Clone Graph (133)

Phase 5: Linked Lists

1. Reverse List (206)
2. Merge Lists (21)
3. Cycle Detection (141)
4. Palindrome (234)

Master the patterns: Backtracking for choices, DFS for depth, BFS for breadth
Remember: Objects are references! Primitives are copies! Capture queue size before loop! Create new list each iteration! Global variables for tracking when returning something else
Practice with small examples, draw trees, trace execution!